📁 Data File Handling in Python

1. Introduction to Files

Why Files?
Data stored in variables/arrays is temporary (lost when program ends).
Files provide permanent storage on disk/SSD. Used for: saving user data, logs, configs, databases, etc.

Types of Files

File Type	Description	Examples
Text File	Human-readable characters (ASCII/Unicode). Each byte = one character. Editable in Notepad/VSCode.	.txt, .py
Binary File	Stores data in binary format (0/1) — not human-readable.	.dat, .bin, .pkl
CSV File	Comma Separated Values — tabular data (rows & columns). Like Excel without formatting.	.csv

2. Relative vs Absolute Paths

Path Type	Description	Example
Absolute Path	Full path from root directory	`C:\\Users\\Student\\Desktop\\file.txt` (Windows - Double back slash "\\" is used because single back slash "\" acts as Escape Sequences: `\t, \n, \r`) `/home/student/file.txt` (Linux/Mac)
Relative Path	Path relative to current working directory	`data/file.txt` or `../file.txt`

3. Text File Handling in Python

Opening a Text File

1. f = open("file.txt", "r") - Traditional Method or Manual Method

```
f = open("filename.txt", "mode") --> filename is compulsory, mode
is optional
```

Characteristics:
- You must explicitly call f.close() to release resources
- If you forget to close, it can lead to resource leaks
- More control but requires manual management

```
f = open("file.txt", "r")    # Manual opening
data = f.read()
f.close()                    # Must close explicitly
```

2. with open("file.txt", "r") as f: - Context Manager Method or With Statement Method

```
with open("filename.txt", "mode") as f: --> filename is
compulsory, mode is optional
```

Characteristics:
- Automatically closes the file when the block ends
- Even if an exception occurs, the file is closed
- Cleaner and more Pythonic (recommended way)

```
with open("file.txt", "r") as f:
    data = f.read()
#(Autocloses) No need to call close()
```

Text File Open Modes

Mode	Description
"r"	Read (default). File must exist otherwise error. File pointer will be at the beginning.
"r+"	Read + write. File must exist otherwise error. File pointer will be at the beginning.
"w"	Write. Creates new file / overwrites existing. File pointer will be at the beginning.
"w+"	Write + read. Creates / overwrites. File pointer will be at the beginning.
"a"	Append. File pointer will be at the end. Creates file if not exists.
"a+"	Append + read. File pointer will be at the end. Creates file if not exists.

Closing a Text File

```
f.close()
```

Writing / Appending to Text File

Method	Description
write(str)	Writes a single string. Returns number of chars.
writelines(list)	Writes list of strings (no automatic newline — add \n manually).

```
with open("out.txt", "w") as f:
    f.write("Hello\n")
    f.writelines(["Line1\n", "Line2\n"])
```

Reading from Text File

Method	Description
read(n)	Reads n characters (or entire file if no argument). Returns the output in `str` type.
readline()	Reads one line (including \n). Returns the output in `str` type. No parameters taken.
readlines()	Returns list of all lines (each as string with \n). Returns the output in `list` type. No parameters taken.

⚠️ **Important:** All three functions returns empty string at EOF.

```
with open("data.txt", "r") as f:
    content = f.read()      # whole file
    f.seek(0)
    line = f.readline()     # one line
    lines = f.readlines()    # list of lines
```

seek() and tell() Methods

tell() → returns current position (bytes from start).
seek(offset, from_what) → moves file pointer.

from_what	Constant	Reference point
0	SEEK_SET	Beginning of file
1	SEEK_CUR	Current position
2	SEEK_END	End of file (offset usually negative)

⚠️ **Important:** In text files, only `seek(offset, 0)` (from beginning) is guaranteed to work. Binary files support all three (0, 1, 2).

```
with open("file.txt", "r") as f:
    print(f.tell())   # 0
    f.read(5)
    print(f.tell())   # 5
    f.seek(0)
    print(f.tell())   # 0
```

Manipulation of Data in Text File

Two approaches: Overwrite method (read all → modify → write back) or temporary file method (safer for large files).

Approach 1: Overwrite Method (Read → Modify → Write Back)

```
# Overwrite example: replace word in the same file
with open("story.txt", "r") as f:
    content = f.read()
new_content = content.replace("cat", "dog")
with open("story.txt", "w") as f:
    f.write(new_content)
```

Approach 2: Temporary File Method (Safer, preserves original)

Example 1: Simple version (Without function)

```
# Simple way using read() instead of loop

# Step 1: Read original and write modified to temp
with open("story.txt", "r") as original:
    data = original.read()

with open("story_temp.txt", "w") as temp:
    temp.write(data.replace("cat", "dog"))

# Step 2: Read from temp file
with open("story_temp.txt", "r") as temp:
    print(temp.read())
```

Example 2: Using function (Advanced)

```
# Method 1 - Easy

# Using a temporary file to avoid data loss

def replace_word_temp(filename, old_word, new_word):
    with open(filename, "r") as original:
        data=original.read()
        print("The old content",data)
        modified_data = data.replace(old_word, new_word)
    with open("story_temp.txt.temp","w+") as temp:
        temp.write(modified_data)
        temp.seek(0)
        print(temp.read())

# Usage
replace_word_temp("story.txt", "cat", "dog")
```

```
# Method 2 - Hard

# Using a temporary file to avoid data loss

def replace_word_temp(filename, old_word, new_word):
    temp_filename = filename + ".temp"

    with open(filename, "r") as original:
        with open(temp_filename, "w") as temp:
            for line in original:
                modified_line = line.replace(old_word, new_word)
                temp.write(modified_line)

    with open(temp_filename, "r") as temp:
        print("Modified content:")
        print(temp.read())

# Usage
replace_word_temp("story.txt", "cat", "dog")
```

File Contents Comparison:

File	Before	After
`story.txt`	"The cat sat on the mat."	"The cat sat on the mat." (unchanged ✅)
`story_temp.txt`	(doesn't exist)	"The dog sat on the mat." (modified)

💡 **Why use temporary file?**

- If the program crashes during writing, the original file remains intact
- Ideal for large files or critical data
- The simple version (read all) works well for small files
- The function version (read line by line) is better for large files

4. Binary File Handling

Opening Modes for Binary Files

Mode	Description
rb	Read binary
rb+	Read + write binary
wb	Write binary (overwrites)
wb+	Write + read binary
ab	Append binary
ab+	Append + read binary

pickle Module — dump() & load()

The `pickle` module converts Python objects (lists, dictionaries, etc.) into a binary stream for storage.

Methods Overview

Method	Parameters	Description
`pickle.dump(data, file_obj)`	**data** - Python object (list, dict, tuple, etc.) to be written **file_obj** - File object opened in binary write mode	Writes the Python object to the binary file
`pickle.load(file_obj)`	**file_obj** - File object opened in binary read mode	Reads and reconstructs the Python object from the binary file

⚠️ **Important:** Both `dump()` and `load()` require file objects opened in **binary mode** (`"wb"`, `"rb"`, `"ab"`, etc.)
`pickle` module should be imported.

Example: Writing and Reading Data

```
import pickle

# Writing data to binary file
with open("data.dat", "wb") as f:
    pickle.dump([1, 2, 3], f)      # Storing a list
    pickle.dump({"name": "Alice", "age": 25}, f)  # Storing a
dictionary

# Reading data from binary file
with open("data.dat", "rb") as f:
    data1 = pickle.load(f)   # Reads the list
    data2 = pickle.load(f)   # Reads the dictionary
    print(data1)    # Output: [1, 2, 3]
    print(data2)    # Output: {'name': 'Alice', 'age': 25}
```

Operations on Binary File (Create, Read, Search, Append, Update)

Operation	How to Perform	Mode
Create/Write	`dump()` to write objects	`"wb"`
Read	`load()` to read objects	`"rb"`
Search	Read all objects, then check condition	`"rb"`
Append	`dump()` to add new objects	`"ab"`
Update	Read all → modify in memory → write back (overwrite)	`"rb"` + `"wb"`

Example 1: Append Operation

```
# Appending a new record to binary file
with open("data.dat", "ab") as f:
    pickle.dump({"roll": 101, "name": "Amit"}, f)
    pickle.dump({"roll": 102, "name": "Priya"}, f)
```

Example 2: Read All Records

```
# Reading all records (knowing there are exactly 3 records)
records = []
with open("data.dat", "rb") as f:
    # If you know there are exactly 3 records
    records.append(pickle.load(f))  # 1st record
    records.append(pickle.load(f))  # 2nd record
    records.append(pickle.load(f))  # 3rd record

for rec in records:
    print(rec)
```

Example 3: Search Operation

```
# Search for a specific record
records = []
with open("data.dat", "rb") as f:
    # Reading all records
    records.append(pickle.load(f))
    records.append(pickle.load(f))
    records.append(pickle.load(f))

# Search for roll number 101
for rec in records:
    if rec.get("roll") == 101:
        print("Found:", rec)
```

Example 4: Update Operation

```
# Update: Read all → modify → write back
records = []
with open("data.dat", "rb") as f:
    records.append(pickle.load(f))
    records.append(pickle.load(f))
    records.append(pickle.load(f))

# Modify record with roll number 101
for rec in records:
    if rec.get("roll") == 101:
        rec["name"] = "Updated Name"

# Write back all records (overwrite)
with open("data.dat", "wb") as f:
    for rec in records:
        pickle.dump(rec, f)
```

5. CSV File Handling

Import CSV Module

```
import csv
```

📌 File modes are same as `text file`.

Writing to CSV File

Method	Description
writer(file_obj)	Returns writer object Takes one parameter (compulsory) - file_object
writerow(row)	Writes one row (list)
writerows(rows)	Writes multiple rows (list of lists)

📌 Both `writerow()` and `writerows()` are functions of `writer` object. `writer()` is a function of `csv` module.

```
with open("data.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["Name", "Age"])
    w.writerows([["Alice", 20], ["Bob", 21]])
```

📌 Always use `newline=""` when opening CSV files on Windows to avoid extra blank lines.

Reading from CSV File

Method	Description
reader(file_obj)	Returns reader object Takes one parameter (compulsory) - file_object

📌 `reader()` is a function of `csv` module.

```
with open("data.csv", "r") as f:
    r = csv.reader(f)
    for row in r:
        print(row)    # row is a list of strings
```

6. Quick Revision

File Modes

File Type	Read Modes	Write Modes	Append Modes	Special Module
Text	r, r+	w, w+	a, a+	—
Binary	rb, rb+	wb, wb+	ab, ab+	pickle
CSV	same as text file	same as text file	same as text file	csv

Common Functions (Available for All File Types)

Function	Description	Syntax
`open()`	Opens a file and returns a file object	`open(filename, mode)`
`close()`	Closes the file and releases resources	`file_obj.close()`
`seek()`	Moves file pointer to a specific position	`file_obj.seek(offset, from_what)`
`tell()`	Returns current position of file pointer	`file_obj.tell()`

Quick Comparison of File Type Specific Functions

Operation	Text File	Binary File	CSV File
Read	`read(n)`, `readline()`, `readlines()`	`pickle.load()`	`csv.reader()`
Write	`write()`, `writelines()`	`pickle.dump()`	`csv.writer()`, `writerow()`, `writerows()`

💡 **Remember:**

- `open()`, `close()`, `seek()`, `tell()` work for **all file types**
- Binary files **must** use `pickle` module for reading/writing objects
- CSV files need `csv` module for proper reading/writing
- Text files use built-in methods directly on the file object

7. Summary of Methods (Quick Glossary)

- `open()` — returns file object.
- `close()` — releases file resource.
- `read()`, `readline()`, `readlines()` — text reading.
- `write()`, `writelines()` — text writing.
- `seek()`, `tell()` — pointer manipulation.
- `pickle.dump()`, `pickle.load()` — binary object serialization.
- `csv.writer()`, `csv.reader()` — CSV handling.
- `writerow()`, `writerows()` — CSV writing utilities.
